# UNIT 10 IoT APPLICATION DEVELOPMENT

**Structure**



## 10.0 INTRODUCTION

In the earlier unit, we had studied various IoT networking and connectivity technologies. After going through the basics of IoT in previous units, we will concentrate on IoT Application Development in this unit.

When you are developing some application, *Platform* is one which allows you to deploy and run your application. A platform could be a hardware plus software suite upon which other applications can operate. Platform could comprise hardware above which Operating system can reside. This Operating system will allow application to work above it by providing necessary execution environment to it.

IoT application platforms provide a comprehensive set of generic, i.e. application independent functionalities which can be used to build IoT applications. When there is only one communication link between devices of one type with another device of same type then, a system of specific service can be set up. But in case of communication among devices of multiple types, there is a need of some common standard application platform which hides heterogeneity of various devices by providing a common working environment to them.

An IoT application platform is a virtual solution, means it resides over cloud. Data is the entity that drives business intelligence and every device has something to talk with other device that is data. By means of cloud connectivity, IoT application platform translates such devices data into useful information. So it provides user means to implement business use cases and enables predictive maintenance, pay-per-use, analytics and real time data management. Thus, IoT application platforms provide a complete suite for application development to its deployment and maintenance.

In this unit we will focus on IoT Application requirements, challenges of IoT Application development, IoT Application Development Frameworks, Open Source platforms for developing IoT applications, Tools for designing and developing IoT application prototypes, IoT application testing strategies and towards the end we will study the security issues in IoT systems.

## 10.1 OBJECTIVES

After going through this unit, you shall be able to:

- understand various requirements for IoT application development;
- list and describe various challenges of IoT application development;
- describe the application development frameworks;
- discuss various types of tools and open source IoT development platforms
- elucidate the testing strategies to be followed for IoT system testing; and
- explain security issues in IoT systems.

## 10.2 IoT APPLICATION ESSENTIAL REQUIREMENTS

The nature of the technology architecture contributes to the essential requirements of IoT applications. Based on the characteristics of the IoT technology ecosystem such as heterogeneity, enormous scale, high volume of data and dynamism, a set of essential requirements for IoT applications is described. These requirements combined with quality attributes can be used to develop a set of high-level requirements for IoT applications. The list is not exhaustive but includes the vitally essential requirements.

### 10.2.1 Adaptability

IoT systems will consist of several nodes, which will be resource constrained mobile and wirelessly connected to the Internet. Due to the factors such as poor connectivity and power shortage, nodes can be connected and disconnected from the system arbitrarily. Furthermore, the state, location and computing speed of these nodes can change dynamically. All these factors can make IoT systems to be extremely dynamic. In a physical environment that is highly dynamic, IoT application needs to be self-adaptive to manage the communication between the nodes and the services using them. IoT applications need to designed and developed in a way that it can efficiently and effectively react in a timely manner to the continuously changing context in accordance with, for instance, business policies or performance objectives that are defined by humans. IoT applications should be self-optimizing, self-protecting, and self-configuring, resilient and energy-efficient.

### 10.2.2 Intelligence

Intelligent things and system of systems are the building blocks of IoT. IoT applications will power IoT enabling technologies in transforming everyday objects into smart objects that can understand and obtain intelligence by making or enabling context-related decisions, resulting in the execution of tasks independently without human intervention. Achieving this requires IoT application to be designed and developed with intelligent decision-making techniques such as context-aware computing service, predictive analytics, complex event processing and behavioural analytics.

### 10.2.3 Real time

A number of IoT domains requires the timely delivery of data and services. For instance, consider IoT in scenarios such as telemedicine, patient care and vehicle-to-vehicle communications where a delay in seconds can have dangerous consequences. Environments, where operations are time-critical, will require IoT applications that provide on-time delivery of data and services.

### 10.2.4 Security

Privacy, trust, confidentiality and integrity are considered important security principles for IoT due to the large number of devices, services and people connected to the Internet. These principles are the top priority and essential requirements for IoT applications. Since the IoT application uses data in various forms, speed and from a variety of sources, it is important it incorporates trust mechanisms that enforce privacy and confidentiality. In addition, IoT application must integrate mechanisms to check for the integrity of data to avoid the erroneous operation of IoT applications.

### 10.2.4 Regulation compliant

IoT applications may collect sensitive personal information about people's daily activities such as detailed household energy usage profile and travel history. Many people consider this information as confidential. When such information is exposed to the Internet, there is a possibility of privacy leakage, and this could affect the privacy of the individual. In order not to violate the privacy of people, IoT applications must be compliant with the privacy requirements established by law such as data protection rules, otherwise, they could be prohibited.

## 10.3 CHALLENGES IN IoT APPLICATION DEVELOPMENT

IoT's application requirements as previously described combined with the inherent qualities of the IoT technology infrastructure makes the development of IoT application, not an easy task. These characteristics create a set of challenges for the IoT application stakeholders as discussed below.

### 10.3.1 Inherently distributed

IoT applications are typically distributed across several component systems. Basically, some IoT application components will be implemented in the cloud/fog. While functionalities such as real-time analysis and data acquisition are implemented in the IoT device, the application components that allow the end users to interact with the IoT system will be implemented, usually as a separate web, mobile or standalone application. IoT applications may also be distributed over a wide and varying geographical are. As they are distributed, the classical approach of a centralised development methodology dealing with all these software components may no longer be applicable. In addition, designing and implementing distributed applications capable of taking consistent decisions from non-centralised resources is not always an easy task.

### 10.3.2 Deep Heterogeneity

One of the major challenges in the realisation of IoT applications is the interoperability among IoT devices using a variety of technologies. IoT applications involve interactions among heterogeneous devices, providing and consuming services deployed in a heterogeneous network (such as fixed, wireless and mobile). This heterogeneity emanates not only from the difference in features and capabilities but also for other reasons such as the manufacturer's and vendors' products and quality of service requirements since they do not always follow the same standards and protocols. Device and communication heterogeneity can make the portability of IoT applications difficult to achieve.

### 10.3.3 Data Management

The data generated from these heterogeneous devices are generally in huge volume, in various forms, and are generated at different speeds. IoT applications will often make critical decisions based on the data collected and processed. Sometimes, these data can be corrupted for various reasons such as the failure of a sensor, introduction of an invalid data by a malicious user, delay in data delivery and wrong data format. Consequently, IoT application developers are faced with the challenge of developing methods that establish the presence of invalid data and new techniques that capture the relationship between the data collected and the decision to be made.

### 10.3.4 Application Maintenance

IoT applications will be executed on distributed systems consisting of millions of devices interacting in rich and complex ways. Since IoT applications will be distributed over a wide geographical area, there are concerns relating to the feasibility of application deployment that supports corrective and adaptive maintenance. The codes running on these devices will have to be debugged and updated regularly. However, maintenance operations present a number of challenges. Allowing devices to support remote debugging and application updates poses significant privacy and security challenges. In addition, interactive debugging may be difficult due to the limited bandwidth of these devices.

### 10.3.5 Humans in the Loop

Many IoT applications are human-centric applications, i.e. humans and objects will work in synergy. However, the dependencies and interactions between humans and objects are yet to be fully harmonized. Humans in the loop have their advantages. For example, in healthcare, incorporating models of various human activities and assisted technologies in the homes of the elderly can improve their medical conditions. However, IoT applications that model human behavior is a significant challenge, as it requires modeling of complex behavioral, psychological and physiological aspects of human nature. New research is necessary to incorporate human behaviors in IoT application design and to understand the underlying requirements and complex dependencies between IoT applications and humans.

### 10.3.6 Application Inter-dependency

An inter-dependency problem may arise when several IoT applications share services from real-world objects. Consider two IoT applications running concurrently in a home: an energy management application for regulating the energy consumption of the electrical and electronic appliances and a health-care application for monitoring the vital signs of the occupants of the house. To reduce the cost of deployment and channel contention, these applications share the information from the sensors in the home. However, integrating both applications is challenging since each application has its own assumptions about the real world and may have no knowledge of how the other application works. For example, the home health care application may detect depression and decide to turn ON all the lights. On the other hand, the energy management application may decide to turn OFF lights when no motion is detected. Detecting and resolving such dependency problems is important for the correctness of operation of interacting IoT systems.

### 10.3.7 Multiple Stakeholders concern

The development of IoT applications involves various stakeholders with different and sometimes conflicting concerns and expectations. The stakeholders of IoT application development include domain expert, software designer, application developer, device developer and network manager. These stakeholders have to address issues that are attributed to the life-cycle phases of an IoT application such as design, implementation, deployment and evolution. The lack of mechanisms to address the concerns of the various stakeholders and the special skill and expertise required by the stakeholders to identify components and to understand the system contributes to the challenges facing IoT application development.

### 10.3.8 Quality evaluation

Since IoT applications are currently being integrated into the daily activities of our lives and sometimes used in critical situations with little or no tolerance for errors and failures, it therefore means that the overall system quality is important and must be thoroughly evaluated to guarantee that it is of high quality before being deployed. However, evaluating quality attributes such as performance is a key challenge since it depends on the performance of many components as well as the performance of the underlying technologies.

## 10.4 IoT APPLICATION DEVELOPMENT FRAMEWORK

Having studied the IoT application development requirements and challenges let us focus on the layered approach of IoT application development framework in this section.

IoT devices are becoming an integral part of organizations, homes, offices, factories, hospitals, and almost everywhere. Today there are billions of IoT devices that are using embedded systems, such as sensors, processors, communication hardware, and other equipment, to send, collect, and act on data without much human intervention. However, IoT is not a simple technology. It is an amalgam of different technologies that work together in harmony. IoT frameworks have a crucial role in the smooth operation of IoT devices.

The fundamental components as shown in *Figure 1* of IoT framework comprises of *Device Hardware (includes sensors, controllers, micro-controllers, and other hardware devices), Device Software* (in*volves written applications to configure controllers and operate them from the remote and do more), Communications/ Connectivity (communication and connectivity mechanisms and protocols), Cloud Platform* and *Cloud Applications* whose details are given below:

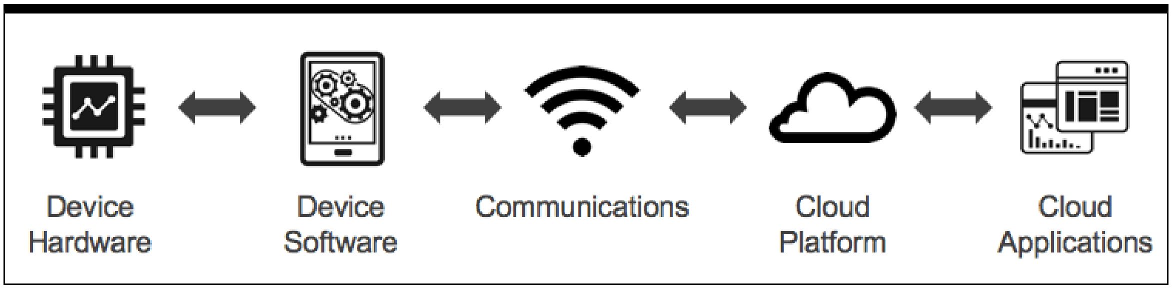


**Figure 1: Framework for IoT Application Development**

### 10.4.1 Device Hardware

Device Hardware is the first layer of IoT technology stack that defines the digital and physical parts of any smart connected product. In this stacked layer, it is imperative to know the implications of size, deployment, cost, useful lifetime, reliability and more such. If we talk about small devices like for example, smartwatches then you may have only one room for such a System on a Chip (SoC). Here, you will need embedded computer like Raspberry-Pi, Artik module, and BeagleBone board.

### 10.4.2 Device Software

The device software is the component that turns the device hardware into a "smart device." Device software is the second layer of the IoT technology stack. Device software enables the concept of "*software-defined hardware*," meaning that a particular hardware device can serve multiple applications depending on the embedded software it is running. It allows you to implement communication with the Cloud or other local devices. You can perform real-time analytics, data acquisition from your device's sensors, and even control.

This layer of the IoT technology stack is critical because it serves as the glue between the real world (hardware) and your Cloud Applications. You can also use device software to reduce the risks of hardware development. Building hardware is expensive, and it takes a lot longer than software. Instead of building your device for a narrow and specific purpose, it is better to use the generic hardware that can be customized by your device software to give you more flexibility down the road. This technique is often known as "software-defined hardware." This way, you can update your embedded software remotely via the Cloud, which will update your "hardware" functionality in the field.

.The device software layer can be distributed into two categories i.e. *Device Operating System* and *Applications*.

#### 10.4.2.1 Device Operating system

The whole complexity of your IoT solution will portray the type of operating system you are in the need of. There are some top things that you must include like when your app requires a real-time operating system, I/O support, and support for the full TCP/IP stack. Some examples of an embedded OS are Brill, Linux, Windows Embedded and VxWorks.

#### 10.4.2.2 Device Applications

Device applications run on top of the Edge OS and provide the specific functionality for your IoT solution. Here the possibilities are endless. You can focus on data acquisition and streaming to the Cloud, analytics, local control, etc.

### 10.4.3 Communications /Connectivity

Communications refer to all the different ways your device will exchange information with the rest of the world. Communications are the third layer of the IoT technology stack. Depending on your industry, some people refer to this layer of the IoT technology stack as *connectivity*. Communications include both physical networks and the protocols you will use. It is true that the implementation of the communications layer is found in the device hardware and device software. But from a conceptual model, selecting the right communication mechanisms is a critical part of your IoT product strategy. It will determine not only how you get data in and out from the Cloud (for example, using Wi-Fi, WAN, LAN, 4G, 5G, LoRA, etc.), but also, how you communicate with third-party devices too.

In the connectivity part of the IoT technology stack, it is important to define the network communication platforms that will be getting connected to the sensors on the product hardware to the cloud and then to the application. The communication part at this stage refers to all the diverse ways where your device will be exchanging information with the whole world. This will include physical networks and the type of protocols that you will be using. It is truly said that the communication mechanisms are connected to the hardware of the device software. Some of the Communication Protocols are -

- Infrastructure (ex: 6LowPAN, IPv4/IPv6, RPL)
- Identification (ex: EPC, uCode, IPv6, URIs)
- Comms / Transport (ex: Wifi, Bluetooth, LPWAN)
- Discovery (ex: Physical Web, mDNS, DNS-SD)
- Data Protocols (ex: MQTT, CoAP, AMQP, Websocket, Node)
- Device Management (ex: TR-069, OMA-DM)
- Semantic (ex: JSON-LD, Web Thing Model)
- Multi-layer Frameworks (ex: Alljoyn, IoTivity, Weave, Homekit)

### 10.4.4 Cloud Platform

The cloud platform is the backbone of your IoT solution. If you are familiar with managing SaaS offerings, then you are well aware of the role of this layer of the IoT technology stack. A cloud platform provides the infrastructure that supports the critical areas like *data collection and management, analytics* and *cloud APIs.*

#### 10.4.4.1 Data Collection

This is an important aspect. Your smart devices will stream information to the Cloud. As you define the requirements of your solution, you need to have a good idea of the type and amount of data you will be collecting on a daily, monthly and yearly basis. One of the challenges of IoT applications is that they can generate an enormous amount of data. You need to make sure you define your scalability parameters so that your architects can determine the right data management solution from the very beginning.

#### 10.4.4.2 Analytics

It is one of the critical component of IoT solution. Analytics refers to the ability to find patterns, crunch data, perform forecasts, integrate machine learning and more. It has the capability to find out the insights from your data that will make your solution valuable. Analytics can be as simple as data aggregation and display or can be as elaborate as using machine learning or artificial intelligence.

#### 10.4.4.3 Cloud APIs

The Internet of Things is all about connecting devices and sharing data, which you can achieve by exposing APIs at either the Cloud level or the device level. Cloud APIs allow your customers and partners to either interact with your devices or to exchange data. Remember that opening an API is not a technical decision; it's a business decision.

### 10.4.5 Cloud Applications

The fifth layer of the IoT technology stack is the *Cloud Applications* layer. Your end-user applications are the part of the system that your customers will see and interact with. These applications will most likely be web-based, and depending on your user needs, you might need separate apps for desktop, mobile, and even wearables. Even though a smart device has its own display, the user may likely use a cloud application as their main point of interaction with your solution. This allows them to have access to your smart devices anytime and anywhere, which is part of the goal of having connected devices. While designing end-user applications, it is very important to understand who your user is and what is his/her primary goal of using the product. The other consideration is that for Industrial IoT (IIoT) applications, you'll probably have more than one user.

Applications can also be divided into *customer-facing* versus *internal apps*. Customer-facing applications usually get the most attention, but in the case of IoT, internal applications are equally important. These include applications to remotely provision and troubleshoot devices, monitor the health of your device fleet, report on performance and predictive maintenance, etc.

These *internal apps* will require a deep understanding of your external and internal customers and will require the right prioritization and resourcing.

In the next section let us study open source platforms and some prototype tools available for IoT Application Development.

## 10.5 OPEN SOURCE IoT PLATFORMS

For understanding an open-source IoT platform, we will ponder on the below three points:

(i) Each consumer desires to utilize any IoT device of their preference without being restricted or bound to a specific product vendor. For example, some smart devices need to be clubbed with only smartphones from the same retailer.
(ii) All business companies of IoT devices desire to integrate their particular devices with ease and diverse ecosystems.
(iii) All application developers desire their apps back multiple IoT devices, which need not demand to blend the specially developed vendor-specific codes.

The open-source framework is a one-stop solution to the above constraints, and it enables scalability and superior levels of flexibility. Many open-source IoT frameworks can be downloaded for free and installed quite straightforwardly across your applications.

### 10.5.1 Popular Open Source IoT Platforms

Following are some of the popular Open Source IoT platforms:

**Kaa**

Kaa IoT Platform is one the most efficient and rich open-source *Internet of Things* cloud platforms where anyone has a free way to materialize their smart product concepts. On this platform, you can manage an unlimited number of connected devices with cross-device interoperability.

You can achieve real-time device monitoring with the possibility of remote device provisioning and configuration. It is one of the most flexible IoT platforms for your business which is fast, scalable, and modern.

**Macchina.io**

Macchina.io platform provide a web-enabled, modular, and extensible JavaScript and C++ runtime environment for developing IoT gateway applications. It also supports a wide variety of sensors and connection technologies including Tinkerforge, bricklets, Xbee, and many others including accelerometers. This platform is able to develop and deploy device software for automotive telematics and V2X, building and home automation, industrial edge computing and IoT gateways, smart sensors, or energy management systems.

### Zetta

Zetta is a server-oriented platform that has been built around NodeJS, REST, and a flow-based reactive programming development philosophy linked with the Siren hypermedia APIs. They are connected with cloud services after being abstracted as REST APIs. People believe that the Node.js platform is best to develop IoT frameworks. These cloud services include visualization tools and support for machine analytics tools like Splunk. It creates a zero-distributed network by connecting endpoints such as Linux and Arduino hacker boards with platforms such as Heroku. Key features are:

- Runs everywhere, including cloud, PCs, or single-board computers.
- Can turn any device into an API.
- Create geo-distributed networks by linking PCs, BeagleBones, and Raspberry Pis with cloud platforms, such as Heroku.
- Optimized to stream real-time, data-intensive applications.
- Supports almost all device protocols.

### DeviceHive

It is yet another feature-rich ***open-source IoT platform*** that is currently distributed under the Apache 2.0 license and is free to use and change. It provides Docker and Kubernetes deployment options and can be downloaded and use with both public and private cloud. It allows you to run batch analytics and machine learning on top of your device data and more. Various libraries, including Android and iOS libraries, are supported in DeviceHive. Key features are:

- Compatible with Java, Python, Node.js, iOS, Android, and other libraries.
- Usable with public, private, or hybrid cloud networking.
- Connects devices via HTTP, WebSockets, or MQTT.
- Offers few deployment options, i.e., Docker, Docker Compose, and Kubernetes.
- Provides rich support for big data analytics.

### Distributed Services Architecture (DSA)

DSA is an open-source IoT that unifies the separate devices, services, and applications in the structured and real-time data model and facilitates decentralized device inter-communication, logic, and applications. Distributed service links are a community library that allows protocol translation and data integration to and from 3rd part data sources. All these modules are lightweight making them more flexible in use. It implements DSA query DSL and has inbuilt hardware integration support.

**Google Cloud Platform**

Developers can code, test and deploy their applications with highly scalable and reliable infrastructure that is provided by Google and Google itself uses it. Developers have to just pay attention to the code and Google handles issues regarding infrastructure, computing power and data storage facility.

Google is one of the popular IoT platform because of: Fast global network, Google's BigData tool, Pay as you use strategy, Support of various available services of cloud like RiptideIO, BigQuery, Firebase, PubSub, Telit Wireless Solutions, Connecting Arduino and Firebase and Cassandra on Google Cloud Platform and many more.

### 10.5.2 Some Tools for Building IoT Prototypes

IoT opened many new horizons for companies and developers working for the development of IoT systems. Many exceptional products have been developed due to IoT app development. Companies providing Internet of Things solution are creating hardware and software designs to help the IoT developers to create new and remarkable IoT devices and applications. Some of the tools to build IoT prototypes and applications are discussed below:

**Arduino**

Arduino is an Italy based IT company that builds interactive objects and microcontroller boards. It is an open-source prototyping platform that offers both IoT hardware and software. Hardware specifications can be applied to interactive electronics and software includes Integrated Development Environment (IDE). It is the most preferable IDEs in all IoT development tools. This platform is easy and simple to use.

**Raspbian**

This IDE is created for Raspberry Pi board. It has more than 35000 packages and with the help of precompiled software, it allows rapid installation. It was not created by the parent organization but by the IoT tech enthusiasts. For working with Raspberry Pi, this is the most suitable IDE available.

**Eclipse IoT**

This tool or instrument allows the user to develop, adopt and promote open source IoT technologies. It is best suited to build IoT devices, Cloud platforms, and gateways. Eclipse supports various projects related to IoT. These projects include open-source implementations of IoT Protocols, application frameworks and services, and tools for using Lua programming language which is promoted as the best-suited programming language for IoT.

**Tessel 2**

It is used to build basic IoT prototypes and applications. It helps through its numerous modules and sensors. Using Tessel 2 board, a developer can avail Ethernet connectivity, Wi-Fi connectivity, two USB ports, a micro USB port, 32MB of Flash, 64MB of RAM. Additional modules can also be integrated like cameras, accelerometers, RFID, GPS, etc.

Tessel 2 can support Node.JS and can use the libraries of Node.JS. It contains two processors, its hardware uses 48MHz Atmel SAMD21 and 580.

MHz MediaTek MT7620n coprocessor. One processor can help to run firmware applications at high speed and the other one helps in the efficient management of power and in exercising good input/output control.

**Platform IoT- IDE**

It is a cross-platform IoT IDE. It comes with the integrated debugger. It is the best for mobile app development and developers can use a friendly IoT environment for development. A developer can port the IDE on Atom editor or it can install it as a plugin. It is compatible with more than 400 embedded boards and has more than 20 development frameworks and platforms. It offers a remarkable interface and is easy to use.

**Kinoma**

It is a Marvell semiconductor hardware prototyping platform. It enables three different projects. To support these projects two products are available Kinoma Create and Element Board. Kinoma Create is a hardware kit for prototyping electronic and IoT enabled devices. Kit contains supporting essentials like Bluetooth Low Energy (BLE), integrated Wi-Fi, speaker, microphone and touch screen. Element Board is the smallest JavaScript-powered IoT product platform.

## 10.6 IoT APPLICATION TESTING STRATEGIES

Testing is very important phase after the application development is completed. The following are the essential types of tests (as shown in Figure 2) recommended for an IoT application.

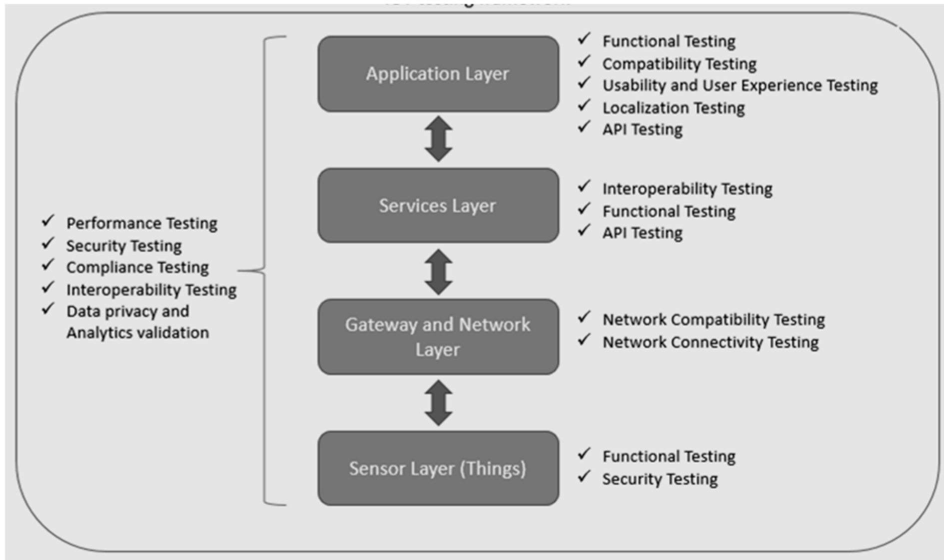


**Figure 2: IoT Application Testing Strategies**

### 10.6.1 Performance Testing

Performing testing is usually conducted so as to determine how rapid the functioning of a communication network model is. This testing also looks into the computation capabilities of the internal part of the software system.

This IoT Performance testing framework is usually done at 3 levels:

- The Network and Gateway level, which involves protocols such as HTTP and MQTT
- The System level
- The Application level

A good example of Performance IoT testing is the verification of response time against a specific bench-marked time, with specifically defined connectivity settings.

### 10.6.2 Security Testing

The security testing aspect of the IoT framework deals with security elements, such as the protection of data, as well as encryption and decryption. It is aimed at providing added security to connected devices, and also to the networks and cloud services on which the devices are connected.

Some variables that mostly cause security threats in IoT are sensor networks, applications that work to collect data, and interfaces. Therefore, it is highly recommended that security testing be done at the device and protocol level, since problems can easily be detected and solved at this level.

An example of security testing is the verification of no unauthorized access to a particular device.

### 10.6.3 Compatibility Testing

The main purpose of compatibility testing is to validate all the possible functional combinations of devices, their hardware, protocol and software versions, as well as operating systems, such as the mobile OS versions. This compatibility testing is usually done in two levels:

- The Application layer
- The Network layer

A good example of compatibility testing is verifying that a particular IoT software supports a given set of devices.

### 10.6.4 End-User Application Testing

The End-user application testing takes into consideration the user experience, as well as the usability and functionality of the IoT application.

An example of this IoT testing framework is the verification of an IoT application, so as to ensure that it includes all required features, and in a good working condition as well.

### 10.6.5 Device Interoperability Testing

This type of testing aims to assess the interoperability of protocols and devices, compared with varying standards and specifications.

In other words, in an IoT framework, the device interoperability testing is conducted so as to verify the connectivity of all devices and protocols.

This testing is usually done in the Service layer. This is because the service layer provides the most conducive environment for this testing, that is; a platform that is communicable, programmable and operable.

## 10.7 SECURITY ISSUES IN IoT

The IoT is diverse from traditional computers and computing devices, makes it more vulnerable to security challenges in different ways:

- Many devices in the Internet of Things are designed for deployment on a massive scale. An excellent example of this is sensors.
- Usually, the deployment of IoT comprises of a set of alike or nearly identical appliances that bear similar characteristics. This similarity amplifies the magnitude of any vulnerability in the security that may significantly affect many of them.
- Similarly, many institutions have come up with guides for risk assessment conduction. This step means that the probable number of links interconnected between the IoT devices is unprecedented. It is

also clear that many of these devices can establish connections and communicate with other devices automatically in an irregular way. These call for consideration of the accessible tools, techniques, and tactics which are related to the security of IoT.

Even with the issue of security in the sector of information and technology not being new, IoT implementation has presented unique challenges that need to be addressed. The consumers are required to trust the Internet of Things devices and the services are very secure from weaknesses, particularly as this technology continues becoming more passive and incorporated in our everyday lives. With weakly protected IoT gadgets and services, this is one of the very significant avenues used for cyber attacks as well as the exposure of the data of users by leaving data streams not protected adequately. The nature of the interconnection of the IoT devices means if a device is poorly secured and connected it has the potential of affecting the security and the resilience on the Internet internationally. This behavior is simply brought about by the challenge of the vast employment of homogenous devices of IoT. Besides the capability of some devices to be able to mechanically bond with other devices, it means that the users and the developers of IoT all have an obligation of ensuring that they are not exposing the other users as well as the Internet itself to potential harm. A shared approach required in developing an effective and appropriate solution to the challenges is currently witnessed in the IoT.

When it comes to authentication, for instance, IoT faces various vulnerabilities, which remain one of the most significant issues in the provision of security in many applications. The authentication used is limited in how it protects only one threat, such as Denial of Service (DoS) or replay attacks. Information security is one of the significant vulnerable areas in the authentication of IoT due to the prevalence of applications which are risky due to their natural multiplicity of data collection in the IoT environment. If we can, for instance, take an example of contactless credit cards. These cards are capable of permitting card numbers and names to be read without the authentication of IoT; this makes it possible for hackers to be able to purchase goods by using a bank account number of the cardholder and their identity.

One of the most prevalent attacks in the IoT is the man in the middle, where the third-party hijack communication channel is aimed at spoofing identities of the palpable nodes which are involved in network exchange. Man in the middle attack effectively makes the bank server recognize the transaction being done as a valid event since the adversary does not have to know the identity of the supposed victim.

In this section let us discuss the **security issues layer-wise** in an IoT Architecture. IoT systems can be broadly described using a basic three layer architecture namely Perception layer, Gateway layer and Cloud layer.

***Perception layer:*** The Perception layer is the typical external physical layer, which includes sensors for sensing and gathering information about the surrounding environment such as temperature, humidity, pressure etc..Table 1 shown below depicts the major threats in the Perception layer:

**Table 1:Threats in the Perception Layer**

| Name of theThreat | Description |
|---|---|
| Denial of Service Attack | IoT sensing nodes have limitedcapacity and capabilities thus attackers can use Denial of Service attack to stop the service. Eventually servers andthe devices will be unable to provide its service for users. |
| Hardware Jamming | Attacker can damage the node by replacing the parts of the node hardware. |
| Insertion of Forged nodes | Attacker can insert a falsified or malicious node between the actual nodes of the network to get access and get control over the IoT network. |
| BruteForce Attack | As the sensing nodes contains weaker computational power brute force attack can easily compromise the access control of the devices. |

***Gateway layer:*** The Gateway layer is responsible for connecting to network devices, interconnected smart devices and servers. Its features are also used for transmitting and processing sensor data. Table 2 shown below depicts the major threats in the Gateway layer:

**Table 2: Threats in the Gateway Layer**

| Name ofthe Threat | Description |
|---|---|
| Denial of Service Attack | As this layer provide network connectivity by following a DOS attack, servers or devices are unable to provide the services to the user. |
| Session Hijacking attacks | Attackers can hijack the session and obtain the access to the network through this kind of attack. |
| Man in the middle (MIM) attacks | Attacker can intersect the communication channel between two sensing nodes and easily obtain classified information if there is no proper encryption mechanism in place. |

***Cloud layer:*** The IoT Cloud Layer represents the back- end services required to set up, manage, operate, and extract business value from an IoT system. It will deliver the application specific services to the user so they can operate and monitor the devices. Following are the threats in the Cloud Layer. Table 3 shown below depicts the major threats in the Cloud layer:

**Table 3: Threats in the Cloud Layer**

| Name of the Threat | Description |
|---|---|
| Data security incloud computing | All the Data that is collectedwill be processed and stored onthe cloud, Cloud service provider will be hold the responsibility of protecting thisdata. |
| Application layerattacks | Most applications are hosted on the cloud as a Software as a Service and delivered through web services, so the attacker can easily manipulate the application layer protocols and get access to the IoT network. |
| An attack on Virtual Machines | Security of cloud virtualmachines is very important and any security breach cancause the failure of entire IoT environment. |

### 10.7.1 Counter Measures

Basic IoT system requires following to be fulfilled in order to become a secure system.

- Authentication
- Authorization
- Confidentiality
- Integrity
- Non Repudiation

Authentication verifies the identity of the users or a device in an IoT system. Authorization checks for what are the privileges possess by the authorized entity to execute on the system. In terms of confidentiality and the data integrity it will make sure that the data is encrypted so no one can tamper even in the storage or during the transmission. Non repudiation assures that authenticity of the origin source of data and integrity of data. Exploiting an IoT system deals with compromising any of the aforementioned security attributes which we need to take actions before compromising.

The Open Web Application Security Project(OWASP), has released latest vulnerabilities that will target the IoT devices and following are the current ranked list of the top issues and things to avoid**:**

- Weak, guessable, or hardcoded passwords
- Insecure network services
- Insecure ecosystem interfaces
- Lack of secure update mechanism
- Use of insecure or outdated components
- Insufficient privacy protection
- Insecure data transfer and storage
- Lack of device management
- Insecure default settings
- Lack of physical hardening

Following table 4 will depict what we can do to improve the security in terms of authentication, authorization, confidentiality, data integrity and non-repudiation security attributes.

**Table 4: Countermeasures to Improve the Security**

| **Security attribute** | **Action** | **Description** |
|---|---|---|
| Authentication | Use security credentials<br>Use identity and access management methods | Identification of users and devices need to be done and need to configure strong security credentials for the devices by removing the default credentials. |
| Authorization | | |
| Confidentiality | Use appropriate encryption mechanism as Devices may contain less computational power | Data must be encrypted so only authorized users can access the data. |
| Data integrity | Use Hashing techniques | Non tampering of data can be assured by various hashing techniques |
| Non repudiation | Using Digital signatures | Origin source of the data can be assured by using digital signatures. |

☞ **Check Your Progress 1**

1) Compare and contrast various IoT platforms discussed in this unit with reference to the parameters like *services availability* and *device management platform*.

………………………………………………………………………………………

………………………………………………………………………………………

………………………………………………………………………………………

2) What are the various factors and concerns those might have an impact on compromising the efforts to secure the IoT devices?

………………………………………………………………………………………

………………………………………………………………………………………

3) Explore and write various current innovative techniques to mitigate the security attacks.

………………………………………………………………………………………

………………………………………………………………………………………

## 10.8 SUMMARY

In this unit we have studied essential requirements for IoT Application development, challenges of IoT Application development, IoT Application Development Frameworks, Open Source platforms for developing IoT applications, Tools for designing and developing IoT application prototypes, IoT application testing strategies and the security issues in IoT systems.

## 10.9 SOLUTIONS / ANSWERS

**Check Your Progress 1**

1. Comparison of various IoT platforms (Open-source) are summarized below in table 5:

**Table 5: Comparison between various open source IoT Platforms**

| IoT Platform | Services | Device Management Platformform |
|---|---|---|
| KAA IoT | Supports Various Hardware Types, Device Management, Reliably Collect Data, Configuration Management, Support Various Integrations, Command Execution, Connect Devices directly or via Gateways | Yes |
| MACCHINA.io | Secure Web Access To IoT Devices From Anywhere, Remote Control Of IoT Devices With Apps and Voice Assistants, Secure Remote Management via Shell and Desktop (VNC & RDP). | Yes |
| ZETTA | Run Everywhere, API Every Thing, Support almost all Device Protocols. | No |
| DeviceHive | Provide End-to-End Solutions, Consulting and Commercial Support, Device Enablement, etc. | No |
| DSA (Distributed Services Architecture) | Provides an open-source Apache 2.0 licensed implementation of a DSBroker written in Dart. | No |

**2.** Given below are various factors and concerns those might impact on compromising the efforts to secure the IoT devices:

**Occasional update:** usually, IoT manufacturers update security patches quarterly. The OS versions and security patches are also upgraded similarly. Therefore, hackers get sufficient time to crack the security protocols and steal sensitive data.

**Embedded passwords:** IoT devices store embedded passwords, which helps the support technicians to troubleshoot OS problems or install necessary updates remotely. However, hackers could utilize the feature for penetrating device security.

**Automation:** often, enterprises and end-users utilize the automation property of IoT systems for gathering data or simplifying business activities. However, if the malicious sites are not specified, integrated AI can access such sources, which will allow threats to enter into the system.

**Remote access:** IoT devices utilize various network protocols for remote access like Wi-Fi, ZigBee, and Z-Wave. Usually, specific restrictions are not mentioned, which can be used to prevent cybercriminals. Therefore, hackers could quickly establish a malicious connection through these remote access protocols.

**Wide variety of third-party applications:** several software applications are available on the Internet, which can be used by organizations to perform specific operations. However, the authenticity of these applications could not be identified easily. If end-users and employees install or access such applications, the threat agents will automatically enter into the system and corrupt the embedded database.

**Improper device authentication**: most of the IoT applications do not use authentication services to restrict or limit network threats. Thereby, attackers enter through the door and threaten privacy.

**Weak Device monitoring**: usually, all the IoT manufacturers configure unique device identifiers to monitor and track devices. However, some manufacturers do not maintain security policy. Therefore, tracking suspicious online activities become quite tricky.

3. Some of the current innovative techniques to mitigate the security attacks are:

**Deploying encryption techniques:** Enforcing strong and updated encryption techniques can increase cybersecurity. The encryption protocol implemented in both the cloud and device environments. Thus, hackers could not understand the unreadable protected data formats and misuse it.

**Constant research regarding emerging threats:** The security risks are assessed regularly. Organizations and device manufacturers developed various teams for security research. Such teams analyze the impact of IoT threats and develop accurate control measures through continuous testing and evaluation.

**Increase the updates frequency:** The device manufacturers should develop small patches rather than substantial updates. Such a strategy can reduce the complexity of patch installation. Besides, frequent updates will help the users to avert cyber threats resources from diverse sources.

**Deploy robust device monitoring tools:** Most of the recent research proposed to implement robust device monitoring techniques so those suspicious activities can be tracked and controlled easily. Many IT organizations introduced professional device monitoring tools to detect threats. Such tools are quite useful for risk assessment, which assists the organizations in developing sophisticated control mechanisms.

**Develop documented user guidelines to increase security awareness:** Most of the data breaches and IoT attacks happen due to a lack of user awareness. Usually, IoT security measures and guidelines are not mentioned while users purchase these devices. If device manufacturers specify the potential IoT threats clearly, users can avoid these issues. Organizations can also design effective training programs to enhance security consciousness. Such programs guide users to develop strong passwords to update them regularly. Besides, users are instructed to update security patches regularly. The users also taught and requested to avoid spam emails, third-party applications, or sources, which can compromise IoT security.

## 10.10 FURTHER READINGS


1. Internet of Things, Jeeva Jose, Khanna Publishing, 2018.
2. Internet of Things - A Hands-on Approach, Arshdeep Bahga and Vijay Madisetti, Universities Press, 2015.
3. IoT Fundamentals: Networking Technologies, Protocols and Use Cases for the Internet of Things, Hanes David, Salgueiro Gonzalo, Grossetete Patrick, Barton Rob, Henry Jerome, Pearson, 2017.
4. Designing the Internet of Things, Adrian Mcwen, Hakin Cassimally, Wiley, 2015.